



School of Computer Science and Engineering

Faculty of Engineering

The University of New South Wales

Benchmarks of SIMD-based Set Operations in Rust

by

Alexander Garrity

Thesis submitted as a requirement for the degree of
Bachelor of Engineering in Software Engineering

Submitted: August 2025

Supervisor: Dr Zhengyi Yang

Student ID: z5363512

Abstract

Sets are a crucial data structure within mathematics and fields deriving from mathematics. Intersection is a fundamental operation on sets. This operation is used heavily in many areas such as databases and thus it becomes important for there to exist fast algorithms for set intersection. As with most algorithms, faster implementations depend on using SIMD instructions.

This project builds on the work of Alex Brown and William Miles in their Honours projects on this.

In this thesis report, we investigate differences with different CPU architectures, a new algorithm based on Gather instructions, comparing CPU and GPU compute on with this algorithm, and a discussion about the programmer experience of using SIMD with Rust.

Acknowledgements

I would like to thank Alex Brown and William Miles for providing an excellent starting point to my part of the project and Zhengyi Yang for providing guidance throughout the project.

Abbreviations

SIMD Single Instruction Multiple Data

MIMD Multiple Instruction Multiple Data

Contents

1	Introduction	1
2	Purpose	2
3	Definitions	3
3.1	SIMD	3
3.1.1	SIMD Vector Width	3
3.2	Sets	4
3.3	Benchmarking	4
3.4	Rust	4
3.5	Set descriptions:	4
3.5.1	Cardinality	4
3.5.2	Density	4
3.5.3	Skewness	5
3.5.4	Selectivity	5
4	SIMD Background Information	6
4.1	CPU SIMD	6
4.1.1	General Information	6
4.2	Hardware Support Now and Going into the Future	9
4.2.1	x86	9

4.2.2	Arm	10
4.2.3	The Effect of Scalable Vector Width	11
4.2.4	The Future of Hardware SIMD	11
4.2.5	Programming Language SIMD Support	11
4.3	GPU SIMD	12
5	Algorithm Background Information	13
5.1	Data Structures	13
5.1.1	Bit-Set	13
5.1.2	Hash-Set	14
5.1.3	Tree-Set	14
5.1.4	Sorted Array	14
5.2	Algorithms	15
5.2.1	2-Set algorithms	15
6	Results From Porting To Arm	22
6.1	Explanation and Results	22
6.2	What does this mean?	23
7	Gather Algorithm	27
8	Cuda Gather	31
9	State of SIMD in Rust	37
9.1	Generic with respect to size	37
9.2	Generic with respect to architecture	39
9.3	ARM Neon intrinsics vs x86 intrinsics	40
9.4	The qualitative experience of using just x86 templates	41

9.5	Rust’s templates and other compile time programming	41
9.6	Foreign function interfaces	42
9.7	Additional points on this topic	43
10	Conclusion	44
10.1	Contributions and findings	44
10.2	Future Work	45
	Bibliography	46
	Appendix 1	47

List of Figures

6.1	Apple M1 Broadcast SIMD width compaision	25
6.2	Intel Core Broadcast SIMD width comparison	25
6.3	Intel Xeon Broadcast SIMD width comparison	26
7.1	Results with varying selectivity at a large amount of sets	30
7.2	Results with varying set count at a large amount of sets	30
8.1	Results with varying selectivity at a large amount of sets	35
8.2	Results with varying set count at a large amount of sets	35
8.3	Results with varying cardinality at a large amount of sets	36
8.4	Results with varying cardinality at a small amount of sets. It is pre- dictably very slow due to overheads	36
A.1	General Intel Core results	48
A.2	General Apple M1 results	48
A.3	Arm Broadcast results with varying set size	49
A.4	Arm Broadcast results with varying skew	49
A.5	Arm Broadcast results with varying selectivity	50
A.6	Arm Broadcast results with varying Density	50
A.7	Intel Xeon Broadcast results with varying set size	51
A.8	Intel Core Broadcast results with varying set size	51

A.9 Gather results against selectivity with a large number of sets	52
A.10 Gather results against set count with a large number of sets	52
A.11 Gather results against set size with a large number of sets	53
A.12 Gather results against selectivity with a small number of sets	53
A.13 Gather results against set count with a small number of sets	54
A.14 Gather results against set size with a small number of sets	54
A.15 Cuda Gather results against selectivity with a large number of sets . . .	55
A.16 Cuda Gather results against set count with a large number of sets . . .	55
A.17 Cuda Gather results against set size with a large number of sets	56
A.18 Cuda Gather results against selectivity with a small number of sets . . .	56
A.19 Cuda Gather results against set count with a small number of sets . . .	57
A.20 Cuda Gather results against set size with a small number of sets	57

List of Tables

Chapter 1

Introduction

In a world where big data has become important, it becomes very useful to make database operations fast. An operation commonly used in databases is 'set intersection'. This report delves into benchmarking and creating new algorithms for SIMD set intersection, specifically in Rust. Chapter 2 explains the purpose of this project. Chapter 3 explains the terms used throughout this report. Chapter 4 goes over the current state of SIMD operations. Chapter 5 provides information about set intersection algorithms from the literature. Chapter 6 discusses the results from porting this project to Arm. Chapter 7 explores a new algorithm based on gather instructions. Chapter 8 brings this algorithm to the GPU. Chapter 9 is a subjective look at the implementations of SIMD in Rust and how it's language design impacts it. Chapter 10 summarises the findings from this project and provides suggestions on where this can be taken next.

Chapter 2

Purpose

This section goes into depth about the state of SIMD on CPUs and GPUs

In other research into this topic, the primary identified use case is database operations.

A more obvious example in databases is whenever a conjunctive query is used (a Boolean AND over multiple elements), one is simply finding an intersection of the data-elements. For the algorithms in this project to be useful for that however, the fields being intersected need to be able to contain a large enough number of elements.

A more specific use case is with a more abstract database (which can include search engines) is to find a set of places where a set of elements all appear. This typically manifests with text and words. One can do this by storing an "inverted index", a set of documents which contain a specific element. To find places where a set of elements all appear, one can then intersect all the sets for the elements are searching for. This is a use case for intersecting many sets together which we will get back to later.

Chapter 3

Definitions

This chapter explains various terms to be used later in the report.

3.1 SIMD

SIMD is short for Single Instruction Multiple Data. It is a type of computer instruction that runs the same operation on multiple pieces of data at the same time. One can think of it as vector operations. Typically, these SIMD operations run at similar speeds to scalar instructions [Fog23].

3.1.1 SIMD Vector Width

SIMD instructions act on vectors a number of bits wide. These "SIMD vector widths" are almost always powers of 2 and in CPUs today are only really 128 bits, 256 bits or 512 bits.

3.2 Sets

Sets are a mathematical concept of a collection of datapoints where each datapoint is unique. Sets can be "intersected" which produces a set of elements common in the starting sets.

3.3 Benchmarking

Benchmarking is of course studying the performance of various things, in this case, algorithms/functions for intersecting sets.

3.4 Rust

Rust is a programming language that has existed in some form since 2012. It's known for high performance whilst ensuring memory safety for security reasons.

3.5 Set descriptions:

3.5.1 Cardinality

The cardinality of a set is number of elements in a set.

3.5.2 Density

The density of a set is the cardinality of the set as a proportion of the total possible elements in a set in that context. It is the cardinality of a set divided by the cardinality of the universal set. Density plays a part in the selection of a data representation (with high densities favouring bit-sets). While it itself isn't as relevant for the performance of different algorithms, it can imply a broader data spread which is relevant.

3.5.3 Skewness

The skewness of 2 sets is the ratio of the cardinalities of the sets. It can impact the relative performance certain algorithms based on their search styles.

3.5.4 Selectivity

The selectivity of each of a collection of sets is the skewness of the intersected set relative to one of the input sets.

Chapter 4

SIMD Background Information

This section goes into depth about the state of SIMD on CPUs and GPUs

4.1 CPU SIMD

4.1.1 General Information

Almost all relatively modern CPU instruction sets have a subset of themselves for SIMD processing. In x86-64 land, there exists the very old MMX and 3DNow!, the slightly less old SSE, and the newer AVX lines of SIMD instruction sets. Arm has Neon and SVE. PowerPC has AltiVec, VMX, and VSX. Mips has MSA. And Risc-V has its vector ISA extension. The big issue with SIMD is that unlike most CPU instructions, often because of data dependencies, compilers rarely automatically compile SIMD code. Compilers are most likely to auto-generate SIMD code from loops in a process called "auto-vectorisation".

Given most programmers don't go out of their way to use special features they compiler doesn't do for free, SIMD use isn't that common. Perhaps the bulk of the SIMD code used in a programmer's programs exists just in `memcpy()` and matrix and vector math

operations used in a higher-level library. The flip side to this lack of adoption in many programs is that one can get significant performance uplifts through using SIMD.

An aside, online you can find these programming language performance benchmarks and in these you often see languages like C++ and Rust vastly outperforming a language like Java. If you look at the code however, you'll find the Java program straight forward and expected [Lew25]. If you look at the C++ or Rust code however, you'll usually find some crazy code concoction that heavily uses SIMD [Mil25].

Given these drawbacks of low adoption, it's worth going back to Flynn's taxonomy to discuss an alternative. While MIMD can be very broad, to compare to SIMD, we can limit to considering MIMD on a single CPU core. CPU cores actually have a lot of smaller components within them. Different operations are done on different "Logic Units" within the CPU. These logic units can operate in parallel meaning that with a memory reading logic unit and an integer arithmetic logic unit, one can read from some memory and do an addition of a different value in the registers at the same time. For greater performance, modern CPU cores contain multiple of the same type of logic units, such as to do arithmetic on 2 variables simultaneously.

There are 2 ways MIMD is used on a single core, if the core has hyperthreading, it can receive 2 (in I think all cases it's just 2) separate streams of instructions at once. Hyperthreading is most useful when the 2 hyperthreads are given instructions that are on different types of logic units. Hyperthreading has been a mainstay on higher performance computer CPUs, although is becoming decreasingly popular with almost all Arm chips foregoing it in favour of efficiency, and Intel following suite with their Lunar Lake lineup of laptop CPUs [Cri23] (the exception of the lack of hyperthreading on Arm is with Huawei on their recent chips [Hua25]).

While programmers are much more accustomed to writing multi-threaded code, hyper-threading MIMD is still a mostly explicit way to increase performance. The other way MIMD is used on a single CPU core is with out of order operations. This is where the CPU will figure out what instructions can be run in parallel and run them across multiple ALUs for greater performance. This is not something controllable at all to

the programmer or compiler but for the CPU to be able to do this more often, the programmer should take care to avoid data dependencies.

An example technique a programmer might employ is "loop unrolling" where instead of doing a reduction operation (such as summing the elements in an array) in a loop 1 element at a time, one could iterate with multiple elements in every stage of the loop acting on their own data, so the CPU could give each of them their own logic units.

```
std::array<long long, 4> counts;
for (size_t i = 0; i < vec.size() - vec.size() % 4; i += 4) {
    counts[0] += vec[i];
    counts[1] += vec[i+1];
    counts[2] += vec[i+2];
    counts[3] += vec[i+3];
}
```

Code sample: Some C++ code demonstrating loop unrolling

Despite the advantages in accessibility of programming and the increased usefulness of MIMD, where SIMD shines is that, due to the possibility of sharing components, a SIMD logic unit can save a lot of silicon compared to the multiple scalar logic units required for similar performance.

Additionally, CPUs store upcoming instructions in cache (an instruction cache) and, by being less instructions, more SIMD instructions can fit in this instruction cache than the number of scalar instructions to be used in MIMD that would yield the same performance.

Compared to dedicated accelerators, CPUs with SIMD can be surprisingly comparable in performance.

In AI for example, while running code on a CPU is far from ideal, it's not THAT bad. An example of which was the requirements for Microsoft's CoPilot Plus PCs of a Neural

Processing Unit NPU of a specific performance. The CPU and GPU of many laptops were together able to outperform the NPU in ai compute (with the CPU providing a notable amount of performance). This leads to the drawback of CPU SIMD against accelerators and that is that CPU SIMD is usually far less efficient than dedicated accelerators. That was why Microsoft required a performant NPU because it wanted it to operate in the background without consuming much power [Mic25].

One can see results of this in mobile vs desktop chips where, as we will get to, SIMD on mobile arm chips is far less developed than on computer chips while mobile chips have more accelerators. The benefit of dedicated accelerators is efficiency at the cost of more silicon used for single purposes - so accelerators are used where power is the biggest limiting factor and SIMD is used more where amount of silicon is the biggest limiting factor.

4.2 Hardware Support Now and Going into the Future

4.2.1 x86

As mentioned prior, x86 has many SIMD instruction sets, with newer versions adding more features and or wider vector widths.

SSE2 is 128 bits and is supported on every 64-bit x86 processor.

SSE3, SSSE3 and SSE4 (all 128 bits) are widely supported on CPUs dating back to 2011.

AVX (still 128 bits) is supported on all but the lowest end CPUs from 2011 onwards and the lowest end CPUs from 2020.

AVX2 was introduced in 2015, introduced 256-bit SIMD and is supported on all but the lowest end CPUs from then till 2020 where all CPUs support it. AVX512 supports 512 bits and is the least supported with high end consumer Intel chips supporting it

between 2017 and 2021, Intel server chips from 2016 onwards and AMD chips from 2022 onwards.

4.2.2 Arm

Arm only has 2 main SIMD instruction sets, NEON and SVE (and SVE2).

Neon only 128 bits wide and is supported on all armv8 chips, meaning all 64-bit arm processors support it (this is dating back to 2013 on iPhones and 2015 on high end Android phones).

Arm SVE however is a very different type of SIMD instruction set. It stands for Scalable Vector Extension and it's selling point is it does not have a fixed size. The programmer requests the native SIMD width at runtime and runs code appropriate for that. The big issue with it however is a severe lack of support. Apple and Qualcomm disable it. MediaTek keeps it enabled but only at the same 128-bit internal width as NEON. In fact, no consumer arm CPU core designs use SVE at a wider vector width.

On the server side, there is better support for SVE with ARM's Neoverse N2, V1 and V2 CPU core designs supporting it (and the Neoverse V2 supports the minorly upgraded SVE2). However, only the V1 supports it with a vector width above 128 bits (256 bits in this case).

The easily strongest example of ARM SVE is with Fujitsu's A64FX chip in their Fugaku supercomputer which supports SVE at 512 bits. We will briefly circle back to this chip later.

Risc-V employs a similar variable width instruction set to SVE, but it is too immature to discuss.

4.2.3 The Effect of Scalable Vector Width

For X86 SIMD programming on unknown hardware using intrinsics, programmers typically, will write 3-4 implementations, a scalar implementation for use on anything, an SSE version, an AVX2 version and sometimes an AVX512 version. Software designed to run on known hardware (like a console or a server) might include less implementations.

That is the issue that variable width SIMD instruction sets are trying to rectify (to reduce that number to just 2, SIMD and scalar) however certain algorithms could perform suboptimally at wider widths.

4.2.4 The Future of Hardware SIMD

The future of x86 SIMD is a mixed bag. As mentioned, AVX512 support was removed from consumer Intel CPUs in 2021. Although it was added to all AMD CPUs in 2022 and then they increased the performance of it in 2024 (in their first implementation it was "double pumped" meaning it would only run every 2 cycles).

Particularly as Qualcomm has moved away from stock arm designs, wider arm SIMD seems to be going nowhere.

But because of Arm's more limited SIMD instructions, the future of SIMD generally is looking negative as Arm gains a bigger foothold in laptop computing.

4.2.5 Programming Language SIMD Support

Traditionally, SIMD instructions were just used with inline assembly. Then, there came SIMD intrinsics which are effectively small often inlined architecture specific functions which just call the requested SIMD instructions. They are supported in languages including C/C++, Rust, Fortran and C#. This is the preferred way to achieve high performance SIMD today.

A newer trend has emerged of building in instruction set independent SIMD libraries into programming languages standard libraries. C++26 will introduce a small SIMD library where a user either uses the native SIMD width (default) or specifies a custom width. C++'s SIMD focusses more on simple computation over manipulating SIMD data. Java has its Vector API which works similarly. Julia and Python NumPy use SIMD quite intensively. More notable for this project of course is Nightly (in development) Rust's `std::simd` where a user specifies a vector width at compile time. We will obviously get to this soon.

4.3 GPU SIMD

One may go as far as to say that GPUs are dedicated SIMD cards: where on a CPU, SIMD instructions are a sideshow, just about everything on a GPU involves SIMD.

NVidia advertises their GPUs as having several thousand "Cuda cores" (2560 in the case of my GTX1080 used in this project). However, looking through a CPU lens, a more accurate number for my GTX1080 is only 20 actual cores (what NVidia calls Streaming Multiprocessors). One can think of this GTX 1080 as having 20 cores and then each of those cores have 4 SIMD logic units which then each have 1024 bit wide SIMD support (NVidia calls this a warp) (and that width is common to all Cuda supporting cards) (this is then the width for 32 32 bit elements which is how they calculate the 2560 Cuda cores) [Gam16].

The interface that Cuda provides is one where every SIMD lane is its own core, but with every "core" in a warp executing the same thing at the same time (with if statements acting as masks that pause the execution of other lanes until the branch is complete).

Chapter 5

Algorithm Background Information

This chapter explains the possible data structures and algorithms found in the research that are used in this project. This section is more succinct than it could be because much of this has been covered in Alex Brown and William Miles' theses.

5.1 Data Structures

5.1.1 Bit-Set

Bit-Sets are a good data structure for high density sets where each bit represents whether a given item is in a set or not. One can very easily use SIMD to intersect with this data structure by using a SIMD bitwise and. For lower densities however, this data structure can be very slow due to requiring a search through every potential element.

5.1.2 Hash-Set

A fairly straightforward representation is a hash-set. Hash-sets are the fastest set representation for adding, removing or checking elements. For the problem of set intersection however, they have much more limited utilities.

A hash-set representation is in theory very good for very skewed sets as one could just iterate through the smaller set and check if that element exists in the larger set.

But one downside of this data structure is that being unordered, the existence of an element in a set provides no information about other the existence of other elements, hashing is not that fast of an operation and moving hash-sets is difficult as they depend on information about the hashing function and data can be scattered.

5.1.3 Tree-Set

Another typical set implementation is as a tree set. Tree sets are $O(\log(n))$ to check for an element. The advantage over a hash-set is that when going through 2 sets, the wherever an item is or would be in a set implies that the elements above a given element in set A, contain all the elements above the given element in set B. And similarly for below an element. However, the data in a tree-set is slightly scattered which makes transferring it across many interfaces difficult. And the data could be scattered.

5.1.4 Sorted Array

The data structure used in this project however is a sorted array. A notable thing in this project is we only care about intersection speed, not insertion time. And so, we can settle with $O(N)$ additions and removals. The advantages to a sorted array are data locality and the simplicity of the data structure allowing for easy moving of sets between interfaces.

Usefully, if we assume evenly distributed elements in sets, one can assume that the

expected positions (whether they exist or if they were to exist where would they be) of subsequent elements in 1 set are close by in another set.

Which allows for linear searching to be feasible.

5.2 Algorithms

One can split set intersection algorithms into 2-Set and K-set for the number of sets being intersected.

5.2.1 2-Set algorithms

Further, we can split this into "linear" algorithms and "search" algorithms.

Linear 2-Set algorithms

Zipper

Perhaps the simplest algorithm is similar to a zipper in that 2 sets are searched through, advancing through 1 until the value in 1 set is at the value in the other set (in which case you add the value to the intersected set) or above the value in the other set (in which case you begin advancing that one instead).

```
while idx_a < set_a.len() && idx_b < set_b.len() {
    let value_a = set_a[idx_a];
    let value_b = set_b[idx_b];

    match value_a.cmp(&value_b) {
        Ordering::Less =>
            idx_a += 1,
```

```
    Ordering::Greater =>
        idx_b += 1,

    Ordering::Equal => {
        visitor.visit(value_a);
        idx_a += 1;
        idx_b += 1;
    },
}
```

Code sample: The main loop of Zipper.

This can be optimised to reduce or eliminate the number of branches. Enhancing performance through allowing the CPU to more easily predict upcoming code, allowing it to not have to throughout pipelines for out of order execution, allegedly up to a 50% improvement [Bro23]

```

while idx_a < set_a.len() && idx_b < set_b.len() {
    let value_a = set_a[idx_a];
    let value_b = set_b[idx_b];

    if value_a == value_b {
        visitor.visit(value_a);
        idx_a += 1;
        idx_b += 1;
    } else {
        idx_a += (value_a < value_b) as usize;
        idx_b += (value_b < value_a) as usize;
    }
}

```

Code sample: The main loop of Branchless Zipper.

BMiss

Drawing on the aforementioned single core MIMD with multiple ALUs, one can increase performance by independently checking 2 or 3 elements ahead. An issue that slows this down is that one must check 2 or 3 elements from 1 set against each of the next 2 or 3 elements from the other set.

```

while left.len() >= S && right.len() >= S {
    if left[0] == right[0] || left[0] == right[1] || left[0] == right[2] {
        visitor.visit(left[0]);
    }
    if left[1] == right[0] || left[1] == right[1] || left[1] == right[2] {
        visitor.visit(left[1]);
    }
    if left[2] == right[0] || left[2] == right[1] || left[2] == right[2] {

```

```

        visitor.visit(left[2]);
    }
    unsafe { bmiss_advance(&mut left, &mut right, S) };
}

unsafe fn bmiss_advance<T: Ord>(left: &mut &[T], right: &mut &[T], s: usize) {
    let l = left.get_unchecked(s-1);
    let r = right.get_unchecked(s-1);

    *left = left.get_unchecked(s * (l <= r) as usize..);
    *right = right.get_unchecked(s * (r <= l) as usize..);
}

```

Code sample: The main BMiss loop and a function it calls.

SIMD Shuffling

SIMD shuffling (of a width N) works by taking N elements from both sets and then performing N shuffles of one of the sets of data and at each step comparing with the data from the other set. Then by elementwise ORing the equalities after every shuffle, we have a list of elements in both sets and then with a lookup table (for smaller Ns), we can get the shuffling instruction that we can correctly shuffle the intersected elements into the resulting set.

```

while i_a < st_a && i_b < st_b {
    let v_a: i32x4 = unsafe{ load_unsafe(ptr_a.add(i_a)) };
    let v_b: i32x4 = unsafe{ load_unsafe(ptr_b.add(i_b)) };

    let masks = [
        v_a.simd_eq(v_b),
        v_a.simd_eq(v_b.rotate_elements_left::<1>()),

```

```

        v_a.simd_eq(v_b.rotate_elements_left::<2>()),
        v_a.simd_eq(v_b.rotate_elements_left::<3>()),
    ];
    let mask = or_4(masks);

    visitor.visit_vector4(v_a, mask.to_bitmask());

    let a_max = unsafe { *set_a.get_unchecked(i_a + W - 1) };
    let b_max = unsafe { *set_b.get_unchecked(i_b + W - 1) };

    i_a += W * (a_max <= b_max) as usize;
    i_b += W * (b_max <= a_max) as usize;
}

```

Code sample: The main Shuffling loop for 128 bit SIMD.

SIMD Broadcast

An alternative to SIMD shuffling is Broadcast which is similar except instead of take N elements from both sets and shuffling to compare, broadcast takes N elements from 1 set and repeatedly takes 1 element from the second set and then broadcasts that element to a full SIMD vector to compare to the other set's data.

```

while i_a < st_a && i_b < st_b {
    let v_a: i32x4 = unsafe{ load_unsafe(ptr_a.add(i_a)) };

    let masks = unsafe {[
        v_a.simd_eq(i32x4::splat(*ptr_b.add(i_b))),
        v_a.simd_eq(i32x4::splat(*ptr_b.add(i_b + 1))),
        v_a.simd_eq(i32x4::splat(*ptr_b.add(i_b + 2))),
        v_a.simd_eq(i32x4::splat(*ptr_b.add(i_b + 3))),
    ]}
}

```

```

    ]};
    let mask = or_4(masks);

    visitor.visit_vector4(v_a, mask.to_bitmask());

    let a_max = unsafe{ *set_a.get_unchecked(i_a + W - 1) };
    let b_max = unsafe{ *set_b.get_unchecked(i_b + W - 1) };

    i_a += W * (a_max <= b_max) as usize;
    i_b += W * (b_max <= a_max) as usize;
}

```

Code sample: The main Broadcast loop for 128 bit SIMD.

An issue with both algorithms is that as N increases, the number of instructions per iteration also increases.

Search Algorithms

Search algorithms differ from linear algorithms in that instead of iterating through all elements, they can search ahead and skip elements.

Gallopings

At this stage of the project, the most important example is called Galloping. Galloping involves "galloping ahead" in powers of 2 to binary search for the desired element once it gallops to an element above it.

This can be made faster with SIMD by replacing the last steps of the binary search with a check over multiple elements.

Galloping can be much faster for high skews. [Bro23]

```
for &target in small {

    let mut offset = 1;

    while offset < large.len() && large[offset] <= target {
        offset *= 2;
    }

    let lo: isize = (offset / 2) as isize;
    let hi: isize = (large.len() as isize - 1).min(offset as isize);

    let base = binary_search(large, target, lo, hi);

    if base < large.len() && large[base] == target {
        visitor.visit(target);
    }
    large = &large[base..];
}
```

Code sample: The main loop of Galloping

Chapter 6

Results From Porting To Arm

Comparing primarily Intel Coffee Lake to Apple Firestorm (M1).

6.1 Explanation and Results

So, what exactly is there to look for when comparing the different architectures, what differences could be expected?

While there is likely to be differences when very closely analysing results to determine why an algorithm performs as it is, is there likely to be any major differences?

As expected, the 128-bit (ARM Neon's width) algorithms performed in similar ways to the 128-bit x86 algorithms. Proportionally to scalar `branchless_merge` algorithm, shuffling and broadcast performed about 2.4x better on the Apple chip and 2.7x on the Intel chip.

The arm results are less reliable due to our Linux virtual machine not supporting the in-depth performance metrics and so we are limited to purely using time.

Is there any serious amount more depth to this? Was this porting exercise a waste?

No.

As previously mentioned, the scalar BMISS algorithm achieves performance speed ups by relying on multiple logic units despite the increased instruction count required as a result of breaking up dependencies.

SIMD algorithms of course do that as well just on a larger scale.

Apple's Firestorm CPU cores are limited to 128-bit SIMD with NEON.

However, there are 4 NEON logic units.

By just removing the flags that guard certain algorithms that don't use intrinsics (notably, the highly performing broadcast algorithm set), one can correctly run algorithms designed with a wider SIMD width in mind than what the hardware can support and in such a way that it should be possible to run 2 or more SIMD units in parallel.

This works to get greater performance out of the Firestorm cores. While not a massive speedup, 256- and 512-bit Broadcast provide on average 11% and 32% greater throughput respectively. An Intel® Xeon® Gold 6342 with actual 512-bit SIMD support for reference achieves a speedup of 69% and 96% respectively relative to its 128-bit results.

An aside with that data, that means that going from 256 bit to 512 bit on the Apple chip with only 128-bit SIMD support yields a 19% uplift in performance where the 512-bit capable Intel server chip provides only a 15% uplift from 256 bits to 512 bits.

Where there is a difference to x86 is that running 512-bit Broadcast on the 256-bit supporting Intel Core i5-8400 results in 40% less performance over the 256-bit algorithm and 7% slower relative to its 128-bit algorithm.

6.2 What does this mean?

Does it not matter which SIMD width is targeted? Not really because the only current platform with various sizes for SIMD without explicit variable sizing is x86 which at

least for this CPU, really requires the native SIMD width for maximum performance.

While I can't find an answer to the count of AVX2 capable logic units on my Intel CPU, one can posit that there is only a single AVX2 Integer unit.

What this does serve to tell us that for this specific situation, a greater amount of SIMD logic units does not make up for a greater SIMD width.

On one hand, it would make sense for x86 CPUs with their rich instruction set to prioritise supporting wider SIMD where on ARM where Neon is only 128 bits wide, to focus on more logic units. This however falls apart when you consider Arm's SVE instruction set is mostly only supported at 128 bits.

From the perspective of wanting to support multiple platforms with SIMD, this state where a 30% speedup can be achieved by targeting a vector width 4x higher than the hardware SIMD width is awful.

Where on the x86 side of things, targeting the SIMD width of system is relatively easy and seems to be easily the most performant option, the optimal algorithm on ARM NEON seems to depend primarily on the count of capable SIMD logic units, information not made available to the programmer. Consequently, the programmer can either hardcode a list of common CPUs or simply pick either targeting modern high-performance cores or older, lower end cores. Notably, many ARM CPUs have multiple types of cores. Due to most ARM SVE implementations also only using 128-bit vectors, targeting the provided SIMD width variable would not even be ideal for many workloads.

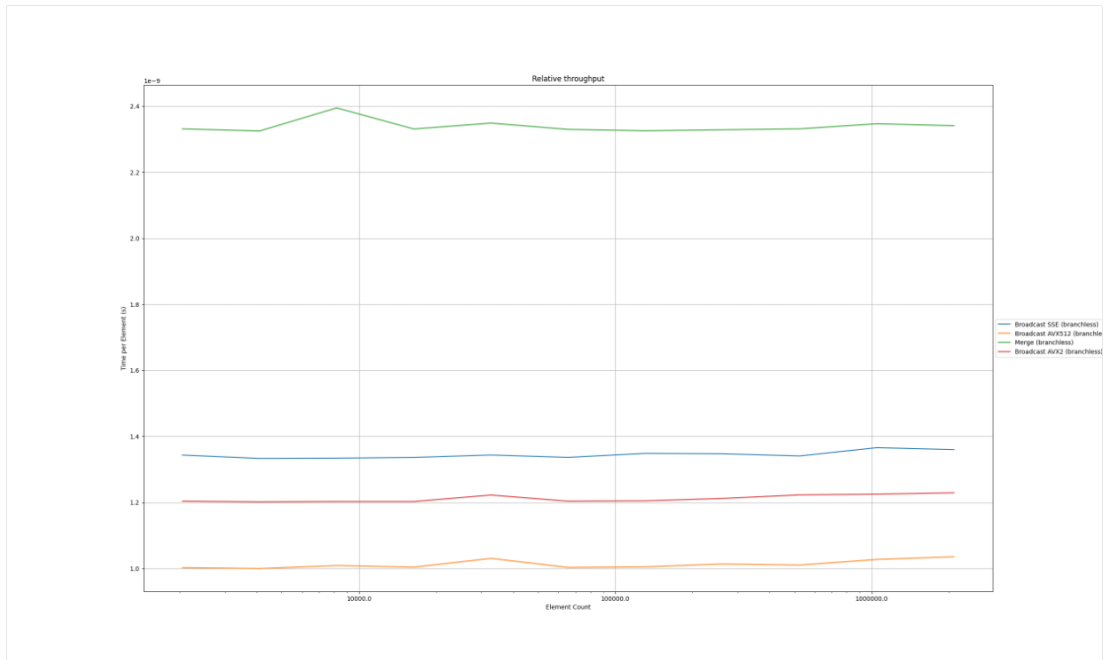


Figure 6.1: Apple M1 Broadcast SIMD width compaiison

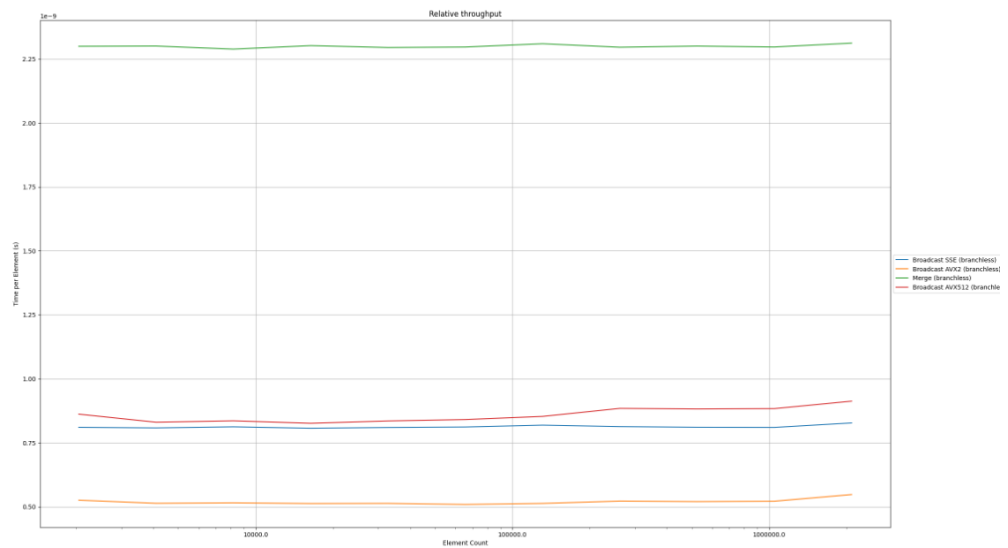


Figure 6.2: Intel Core Broadcast SIMD width comparison

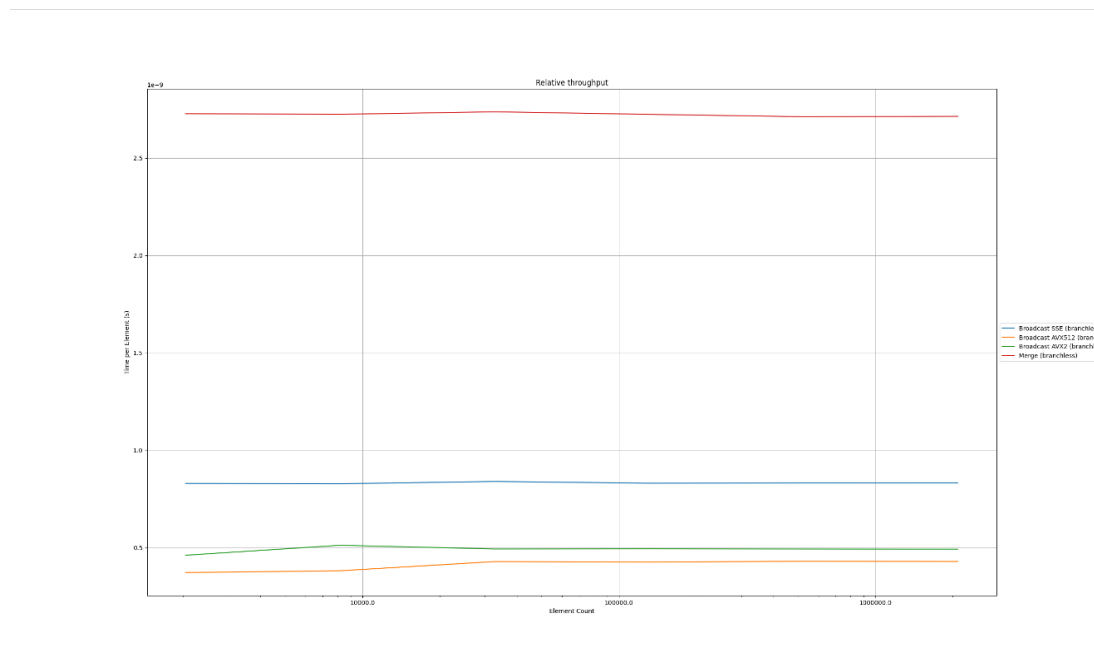


Figure 6.3: Intel Xeon Broadcast SIMD width comparison

Chapter 7

Gather Algorithm

Gather SIMD operations are simply SIMD operations to read memory from a vector of pointers or offsets from an array. They (and their counterpart for writing memory, Scatter) are not well supported across architectures with Arm Neon not properly featuring them. Gather operations are slow, but particularly for uncached memory. Loads from cached memory are still slower than sequential loads but not substantially so.

In some ways, gather and scatter instructions can allow the running of branchless algorithms in parallel on a single core.

The one reference to gather operations we could find within the context of SIMD set intersection was an old paper lamenting the lack of Gather instructions in x86 at the time. [BS11] Where 2-Set intersections have too many dependencies on other data to effectively parallelise, K-set algorithms provide a much more optimistic situation.

We therefore developed an algorithm to take advantage of this (that we uncreatively will just call "Gather"), that runs with AVX2 (because that's the CPU we have and 128-bit SIMD should provide a significantly smaller speedup) is effectively like a 9-way zipper algorithm.

We take 1 set to use as a check set and then take another 8 sets. This algorithm then advances all 8 sets until either the value from 1 of them exceeds the current value in the

check set (in which case the check set is advanced) or until they are at the same value of the check set where they wait for the other sets to get there too. If all 9 sets point to the same value, that value is added to the result set and the algorithm continues until 1 set is exhausted.

By doing this 9-way zipper, one can avoid any scatter instructions which should be much slower.

```
unsafe {
    use std::arch::x86_64::*;

    let mut pointersArr = vec![0; 8];
    let mut pointersArrEnd = vec![0; 8];
    for j in 0..8u32 {
        pointersArr[j as usize] = additional.getRange(index + j).start;
        pointersArrEnd[j as usize] = additional.getRange(index + j).end;
    }

    let mut indexes = _mm256_loadu_si256(pointersArr.as_ptr().cast());
    let ends = _mm256_loadu_si256(pointersArrEnd.as_ptr().cast());

    let mut i = 0;
    let offset = additional.getInitial();
    let end = initial.len() as u32;
    let mut resultsVec: Vec<i32> = vec![];
    while (i < end && _mm256_movemask_epi8(_mm256_cmpgt_epi32(ends, indexes)) ==
0xFFFFFFFFFu32 as i32) {
        let val = _mm256_set1_epi32(*initial.get_unchecked(i as usize));
        let vals = _mm256_i32gather_epi32::<4>(offset, indexes);
        if (_mm256_movemask_epi8(_mm256_cmpeq_epi32(val, vals)) ==
0xFFFFFFFFFu32 as i32) {
            resultsVec.push(*initial.get_unchecked(i as usize));
            i += 1;
        }
    }
}
```

```

        indexes = _mm256_add_epi32(indexes, _mm256_set1_epi32(1));
        continue;
    }
    if _mm256_movemask_epi8(_mm256_cmpgt_epi32(vals, val)) != 0 {
        i+=1;
        continue;
    }
    let cmp = _mm256_and_si256(_mm256_cmpgt_epi32(val, vals),
        _mm256_set1_epi32(1));
    indexes = _mm256_add_epi32(indexes, cmp);
}
initial = resultsVec;
index += 8;
}

```

Code Sample: The main part of the gather algorithm

While for very low selectivities, this algorithm can perform well against other linear algorithms (2.3x faster than Broadcast), for most selectivities it is between 70 and 150% slower than Broadcast.

Sadly, low selectivities is also where galloping does very well. At 0.001 selectivity for example, Galloping outperforms gather by 11 times.

Interestingly, while for smaller sizes of the sets being intersected, gather performs linearly with respect to size, with sets beyond somewhere between 8000 and 16000 elements in size (on my machine at least), gather jumps from being about 2.5x slower than broadcast to about 3.2x slower. My presumption is that by requiring the keeping of more arrays in memory, it becomes easier to run out of cache and it i's harder for the CPU to predict where to cache.

Given these very middling results, we do not believe it is worthwhile to go into more depth into this algorithm.

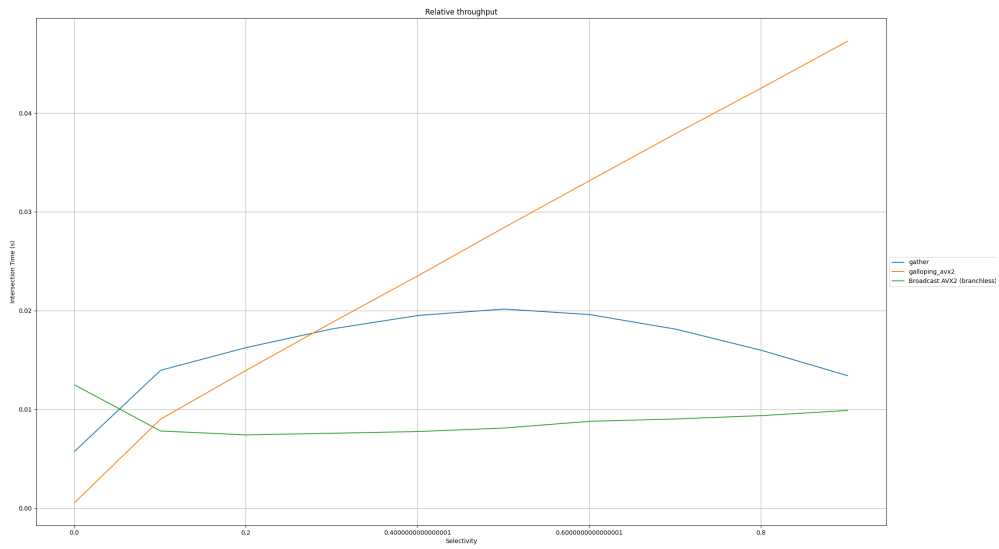


Figure 7.1: Results with varying selectivity at a large amount of sets

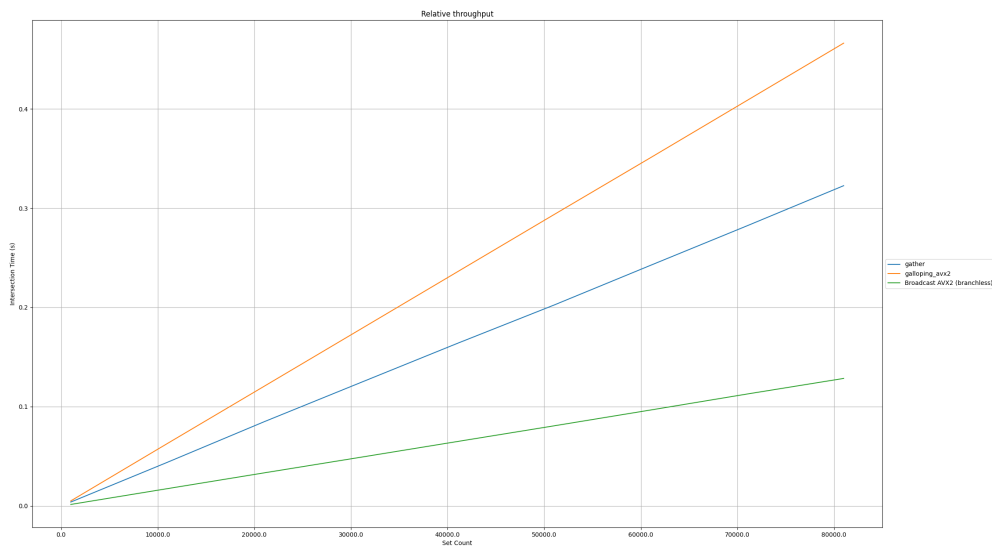


Figure 7.2: Results with varying set count at a large amount of sets

Chapter 8

Cuda Gather

Given Cuda’s very wide effective SIMD width, I did not expend resources attempting to port any 2-set specific algorithms. Instead, I took my gather algorithm and, for a single warp, I extended it to work with 1 check set and 32 additional sets. I then made it work on all the warps on the GPU in parallel in a divide and conquer style approach.

Each layer in that divide and conquer was orchestrated by the CPU.

Because Cuda is designed to be reasonably high-level, I could just specify however many ”blocks” (in this case the 32 Cuda cores) and if it were more than my GPU physically had it would just run them sequentially and would just run on less blocks if I requested less.

```
while (true) {  
    T value = mainData[mainIndex];  
    while (currIndex < end && mainData[currIndex] < value) {  
        ++currIndex;  
    }  
    bool withinRange = currIndex < end;  
    auto mask = __activemask();
```

```

bool withinAll = __all_sync(mask, withinRange);
if (!withinAll || mainIndex >= mainEnd) {
    return;
}

bool eq = mainData[currIndex] == value;
mask = __activemask();
if (__all_sync(mask, eq)) {
    __syncthreads();
    if (threadIdx.x == 0) {
        outputData[outputRanges[blockIdx.x].end] = value;
        ++outputRanges[blockIdx.x].end;
        ++mainIndex;
    }
    __syncthreads();
    ++currIndex;
} else {
    if (threadIdx.x == 0) {
        ++mainIndex;
    }
}
}

```

Code sample: The main loop on the GPU

```

do {
    cuda_warp_gather<<<blockCount, 32>>>(data1, ranges1, SIZE, data2, ranges2, outputSize)
    cudaDeviceSynchronize();
    auto copied = cudaMemcpy(&cpuOutputSize, outputSize, sizeof(size_t), cudaMemcpyDeviceToHost);
    std::swap(data1, data2);
    std::swap(ranges1, ranges2);
}

```

```
        cudaMemcpy(ranges2, outputRangesBase, cpuOutputSize*sizeof(range), cudaMemcpyD
        cudaDeviceSynchronize();
        SIZE = cpuOutputSize;
        blockCount = std::ceil(SIZE/static_cast<double>(33));
    } while (SIZE > 1);
```

Code Sample: The loop that orchastrates the GPU code

We took one look at Rust Cuda libraries and elected to write the CPU code that orchestrates the GPU in C++ that was compiled with NVCC and then used in a foreign function interface in Rust. It is clear Cuda is designed to be called from C++ code.

The system we are running these benchmarks on can be thought of as an unbalanced (in favour of a powerful GPU) mid-high end gaming PC. A GTX 1080 and a Hexa-core Intel Core i5 8400. This almost the best-case hardware loadout for GPU compute to win against CPU compute.

The initial plan was to build some system to run the CPU algorithms on multiple threads for a fair comparison to the GPU. This was not necessary.

For large counts of sets (above a few thousand) and large set sizes (several hundred elements) (where the costs of setting it up are proportionally small), this Cuda algorithm performs between the level of single core CPU Broadcast and single core CPU Gather. (about 50% of the throughput of Broadcast and about 125% of the throughput of CPU gather).

Interestingly, unlike CPU gather, this GPU gather performs exceptionally similarly across different selectivities. Being the most stable algorithm tested.

Yet again, we do not believe it is worthwhile to go into more depth about these performance metrics given this falls very short of alternative algorithms.

Some asides on the experience of programming in Cuda, because one can't easily copy

data structures with any pointer indirection between the CPU and GPU (since the pointers point to memory on different devices), where C++ started as C with Classes, Cuda feels like C++ without classes.

Programming Cuda on a headless server allowed me to run into a frustrating issue where we didn't realise that the GPU wasn't responding due to driver corruption. This is a case where I believe exceptions are superior to errors as values because we didn't check the value that said the reason why copying memory to the GPU wasn't working, a fairly exceptional circumstance.

With Cuda's abstractions and having to copy memory and orchestrate function calls on another device, I am jealous of that 512-bit SVE supporting Fujitsu chip I mentioned earlier, which was designed to be able to function as a GPU and/or CPU.

Though perhaps I should have instead done this in OpenCL on my Mac with its unified memory. (and then have been able to directly compare the performance since the CPU and GPU are from the same chip, take up similar amounts of silicon and consume similar amounts of power).

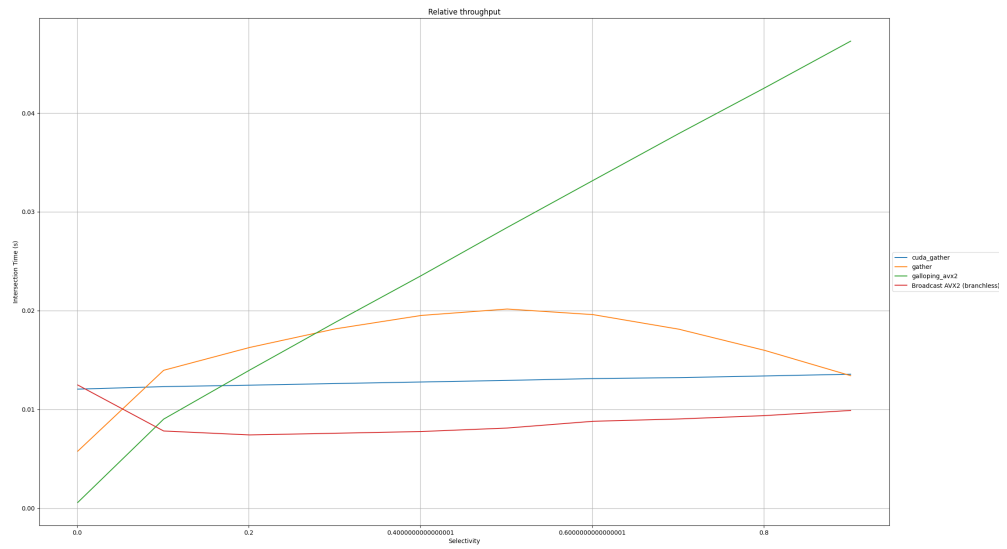


Figure 8.1: Results with varying selectivity at a large amount of sets

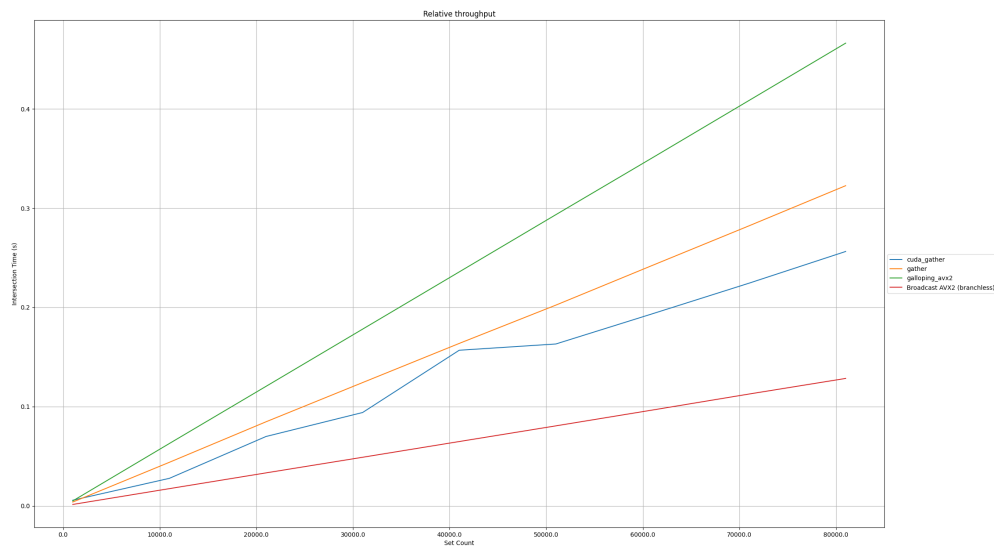


Figure 8.2: Results with varying set count at a large amount of sets

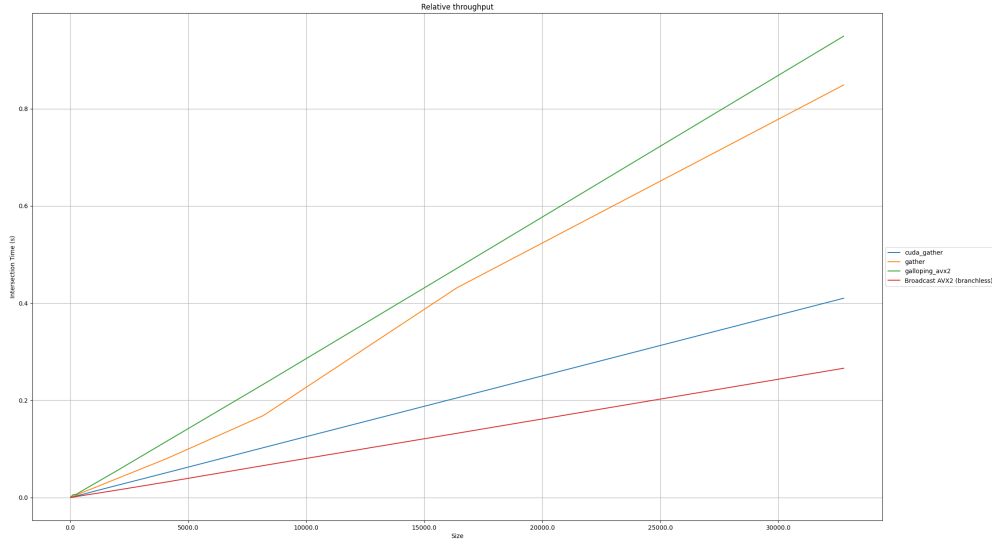


Figure 8.3: Results with varying cardinality at a large amount of sets

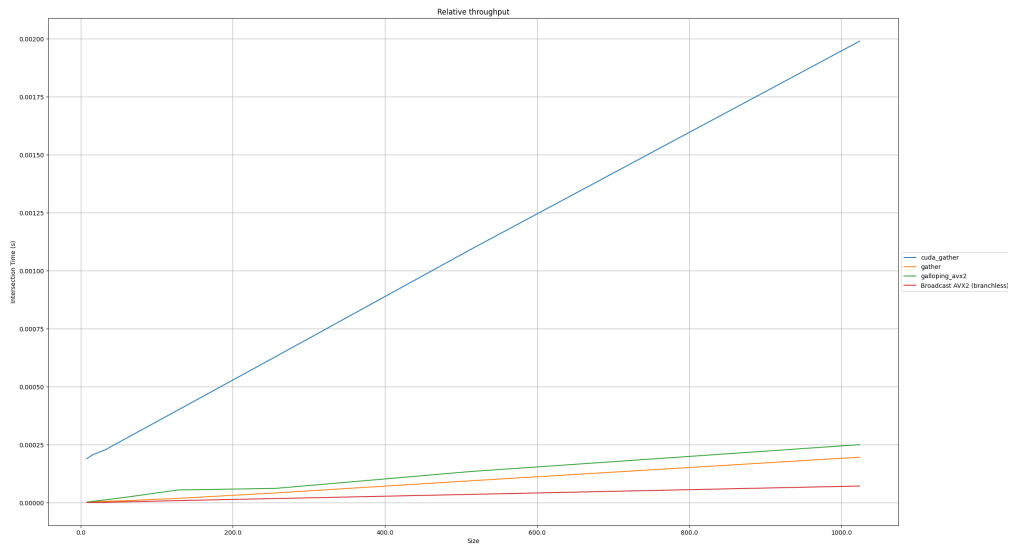


Figure 8.4: Results with varying cardinality at a small amount of sets. It is predictably very slow due to overheads

Chapter 9

State of SIMD in Rust

This is a much more subjective section of this report and such I will be using "I" not "we".

I was informed that the reason why this project was chosen to be in Rust was because, unlike other the other notable high-performance competitors of C and C++ (yet), Rust has a built-in generic SIMD implementation.

This would seem to be very important, specifically as SIMD usage is relatively rare.

However, Rust's Portable SIMD library in reality is quite lacking for many workloads such as in this project.

There are largely 2 benefits for generic SIMD in theory, generic with respect to SIMD vector size and generic with respect to the instruction set.

9.1 Generic with respect to size

Due to the updates to x86's SIMD support over time, notably increasing the maximum vector width, it is very common for people programming with SIMD instructions to use feature flags to indicate to fully support older and newer CPUs by coding versions that

rely on their differing vector widths. Rust's SIMD seeks to eliminate this by allowing programmers to target a specific size (that is programmer specified, one can't find the native vector size of the hardware at runtime or compile time in a generic manner) and then the compiler generates the lower vector width (including scalar) versions.

Rust's approach to this differs from other approaches like Java's Vector API and Arm's SVE (untested in this project due to lack of hardware support) which use a variable length that is given at runtime. A variable length approach would allow the construction of very generic algorithms that, depending on the algorithm, should perform acceptably.

This difference is particularly notable in this project in that most algorithms here do not scale linearly with respect to vector width; when the vector width is widened, more instructions need to be run to account for checking the additional items. This means that if you were to target a high vector width, devices that do not support that vector width would have to run the same heavier algorithm but without any benefits.

A specific example is for the Broadcast algorithm where if 128-bit SIMD is directly targeted, 4 broadcast instructions are needed to iterate through 4 elements. If 256-bit SIMD was targeted on a CPU with native 256-bit SIMD, 8 broadcast instructions are needed to iterate through 8 elements.

However, if that 256-bit version is run on a CPU with only 128-bit SIMD, then the broadcast instructions would need to be split in 2 meaning 16 broadcast instructions per 8 elements.

A variable length approach could still have issues. While between 128-bit SSSE3 and 256-bit AVX2, I have not seen any performance regressions for going with a larger size, I can hypothesise that AMD's Zen 4 architecture where AVX512 instructions take twice the time as AVX2 instructions, would show a similar performance regression to emulated fixed size SIMD when targeting the maximum SIMD size.

9.2 Generic with respect to architecture

A second benefit to a generic SIMD implementation is being able to run on different instruction sets. The inability to get the native vector width in a generic manner requires some amount of hardware specific flags or detection to avoid the issues with SIMD widths. Although it isn't too difficult to simply check like "sse3 or neon or alvec or lasx" (or just "sse3 or neon" [RISC-V SIMD doesn't seem to have intrinsics in Rust yet]) despite this working for variable length SIMD instruction sets such as ARM SVE or RISC-V-V.

An aside, ARM SVE just isn't supported in Rust, even in intrinsic form. Granted, SVE has limited support and usefulness but still.

There are effectively 2 kinds of portable SIMD in rust, `std::SIMD` and `std::intrinsics::SIMD`.

`std::SIMD` is a library in nightly Rust that aims to be truly generic. It is however reasonably limited, lacking basic features such as element-wise equality.

The second type, `std::intrinsics::SIMD` is "unstable". Its use requires a declaration at the main file and offers the bare minimum guarantees about what the functions do. Its element-wise SIMD equality function for example merely guarantees the result is zero or not zero, meaning that the resulting value is difficult to use effectively. And the docs do not say which intrinsics they are using on each platform and so it is more difficult to understand within the context of known targeted platforms.

Altogether, the feature-set of Rust's portable SIMD is limited. While lacking mostly unique features like STTNI for string comparisons is expected, Rust lacks support for important instructions like shuffling/swizzling based on a variable input: something supported in at least modern x86 and ARM.

9.3 ARM Neon intrinsics vs x86 intrinsics

While types at an assembly level do kind of exist (in the form of SIMD registers), they only matter for the class of instructions used and their size. C and C++ have types yet to give the programmer greater choice, allow the programmer to effectively ignore the type of an object by way of easy type casting (C's pointer casting and C++'s `reinterpret_cast` are more relevant here). Rust is very type safe and only supports `reinterpret casts` if it knows the types have the same size.

x86's SIMD intrinsics mimic assembly in that, while there are different types for different classes of types (integers, floats, doubles) but are independent of the size of the variables inside the SIMD vector, just that they add up to a specific size all together). (`_mm128`, `_mm128d`, `_mm128i`, `_mm256`, `_mm256d`, `_mm256i`...)

ARM's intrinsics on the other hand specify the makeup of the vector internally (`uint8x1_t`, `int4x32_t`...). Clearly, there are situations where, as an example, shuffling 4 32-bit integers is a thing someone would want to do. The issue with this is that there isn't an intrinsic that takes in a vector of 4 32-bit integers to shuffle, only 16 8-bit integers. This is a problem because the Rust types for a specific SIMD arrangement can convert to their respective ARM intrinsic type, they can only auto convert to their respective ARM intrinsic type, despite certain instructions requiring specific, different, types.

The consequence of this for supporting multiple platforms in Rust is that where a templated function that calls an x86 intrinsic can support easily support whatever SIMD types are of the same size, a corresponding ARM function is much harder to write.

Thankfully, `std::mem::transmute` exists to just reinterpret the memory to the desired type. Unfortunately for me, this requires the types to be the same size and critically, requires that to be known to the compiler (Unsafe Rust seems to be too safe for me in this situation). Where clauses in templated functions in Rust can't specify a specific size and so it can't be a requirement of the function. Where in C++, the feasibility of a templated function is tested when a concrete type is inputted into it, and so in C++

so long as the inputted types are the same size, the function would work. Not the case in Rust. And so, what I can see is that a function generic to all the possible ARM intrinsics of a certain size seems impossible and so the types must be specified instead. I was quite surprised to learn that function overloading does not exist in Rust. The solution I eventually settled on was to create a "Neon128" struct and then implemented conversions between the various Rust SIMD types and ARM intrinsic types. This does allow for the same function call to work on x86 and ARM which a very important goal for multi-platform SIMD.

9.4 The qualitative experience of using just x86 templates

Surprisingly, given a known width, x86 SIMD intrinsics are quite easy to use. They are all neatly organised as static functions, they are well documented (they're better if you use the Intel name instead of the potentially different rust name) and they don't depend on specific types.

Quite frankly, if I were just dealing with a single platform to target, I would prefer to just write the multiple versions for different levels of SIMD support with intrinsics than use a mix of Rust features.

9.5 Rust's templates and other compile time programming

one thing that slowed me down a lot was the, comparative to C++, severely underdeveloped generic programming.

several missing features of rust generics that became apparent when attempting to write a generic with respect to size SIMD set intersection function is, as previously mentioned,

- the inability to require a certain size of a variable at compile time: the best solution to this I found was to have a runtime check (I could install and use some crate, but I feel as though code relying on external crates is harder to understand).
- that constant variables can't be dependent on templates, whether they be type templates or const value templates, which is significantly limiting for lookup tables.
- the lack of partial specialisation: this significantly impedes the ability to cleanly create a generic SIMD implementation specific to the functions and architecture since it means that instead of being able to fully pick out the correct function at compile time based on a generic size parameter, one would have to have a runtime if statement that can hopefully be successfully branch predicted almost 100% of the time.

while not as much of a roadblock, where C++'s duck typing templates are also quite a bit simpler and faster to write given, unlike with a more object-oriented field, the envisioned types, largely being from the standard library, were not created with this precise usage in mind and so aren't as neatly categorised and so implicit templates can have advantages.

while rust has some amount of generic SIMD, I would argue the currently best solution for multiplatform SIMD is constructing a minimal portable SIMD library for the functionality that's used in a specific project and with preconditions based on the list of platforms being targeted.

9.6 Foreign function interfaces

While it in many ways is an unfair advantage being derived from C, C++ strikes me as a vastly better language for C based foreign function interfaces because it integrates much better with them, where the elements of unsafe rust that deal with foreign function interfaces seem disconnected from the rest of the language (such as pointers simply being invisible to the borrow checker) (I would have presumed you'd instead just have to ignore the borrow checker with unsafe when you would want to do that).

This also poses another benefit of C++ templates for this kind of programming, the easy ability to have a function work with a standard library class, and a basic C style struct.

9.7 Additional points on this topic

the rust documentation is bad for non-x86 intrinsics with zero description about what functions do which, while it doesn't stop multiplatform SIMD, makes it that bit slower to write.

Chapter 10

Conclusion

Ultimately, I feel surprisingly little has been achieved over the course of my work on this project. Learning the code-base, porting to Arm and learning and integrating Cuda have proven to be large time-sinks that have provided comparatively little results. Nevertheless, I have still made contributions and findings.

10.1 Contributions and findings

I have ported the benchmarking suite to Arm and non-Linux which has allowed the discovery that while x86 chips tend to perform best when targeting the largest supported SIMD width, at least some Arm chips can perform better when targeting a wider width - but this results in a worst programmer experience.

I have developed a "gather" based set intersection algorithm for K-Set intersection, yet it performs poorly.

I have then ported this algorithm to Cuda to run on a GPU, to particularly middling results.

But what I feel is the most worthwhile finding from this part of the project was a

discussion on the state of SIMD in Rust, finding their Nightly portable-SIMD library to be incomplete and providing a poor developer experience, and finding some of Rust's design decisions regarding Generics to be unsuited for this kind of SIMD work.

10.2 Future Work

Future work into this area could be spend on the laborious work of finding realistic intersection data to provide the most utility, developing more K-set intersection algorithms including on GPUs. However, I would recommend against continuing on with this project due to this format of an honors thesis encouraging writing code more quickly to hit deadlines at the cost of future maintainability, given each person is working individually. The result of this is that each git branch of the project code as left by Alex Brown, William Miles and myself contains a lot of tech debt. I believe that if this space is to be investigated again, it would be better to start fresh in C++, with a more clearly defined and simpler interface, while continuing to put the SIMD intrinsics that are used behind small functions to be able to more easily port the project to whatever architectures are desired. And unlike the current project only gate functions behind abstracted SIMD widths no specific feature levels.

Bibliography

- [Bro23] Alex Brown. An open experiment in simd-based set intersection. Master's thesis, UNSW, 2023.
- [BS11] Wolfgang Lehner Benjamin Schlegel, Thomas Willhalm. Fast sorted-set intersection using simd instructions. *ADMS*, 2011.
- [Cri23] Michael Crider. Intel abandons hyperthreading for lunar lake cpus. *pcworld*, 2023.
- [Hua25] Huawei. *Feature Differences Between Arm and x86 Servers*, 2025.
- [Lew25] Mark C. Lewis. n-body java naot 4 program. *benchmarksgame-team*, 2025.
- [Mic25] Microsoft. *Copilot+ PCs developer guide*, 2025.
- [Mil25] Miles. n-body rust 9 program. *benchmarksgame-team*, 2025.

Appendix 1

This section contains all the graphs produced for this project.



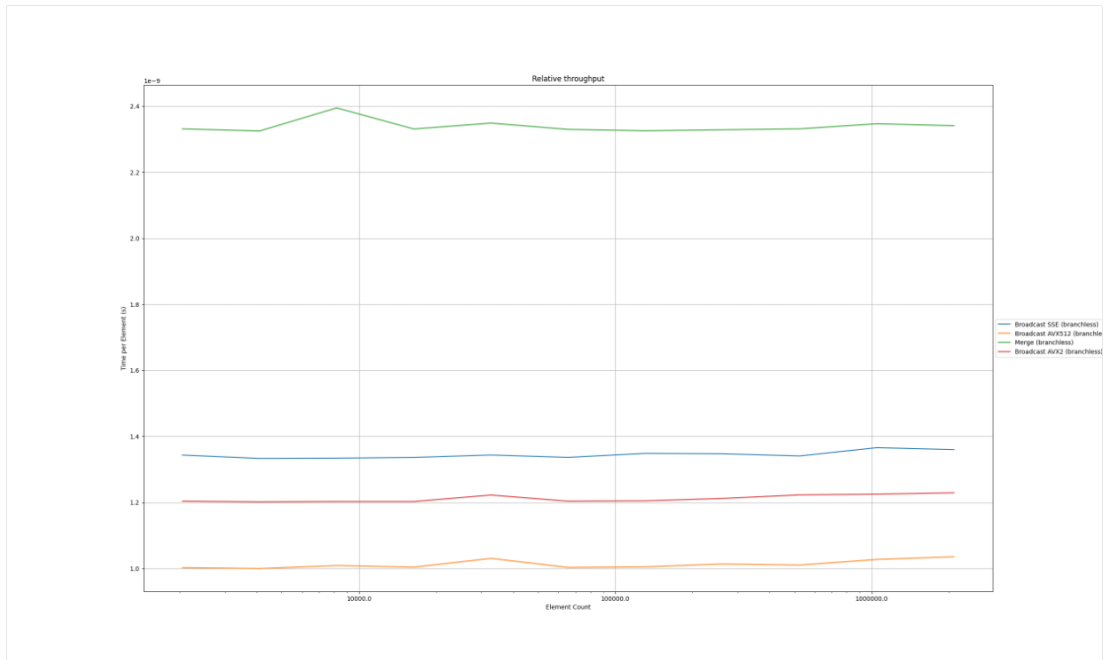


Figure A.3: Arm Broadcast results with varying set size

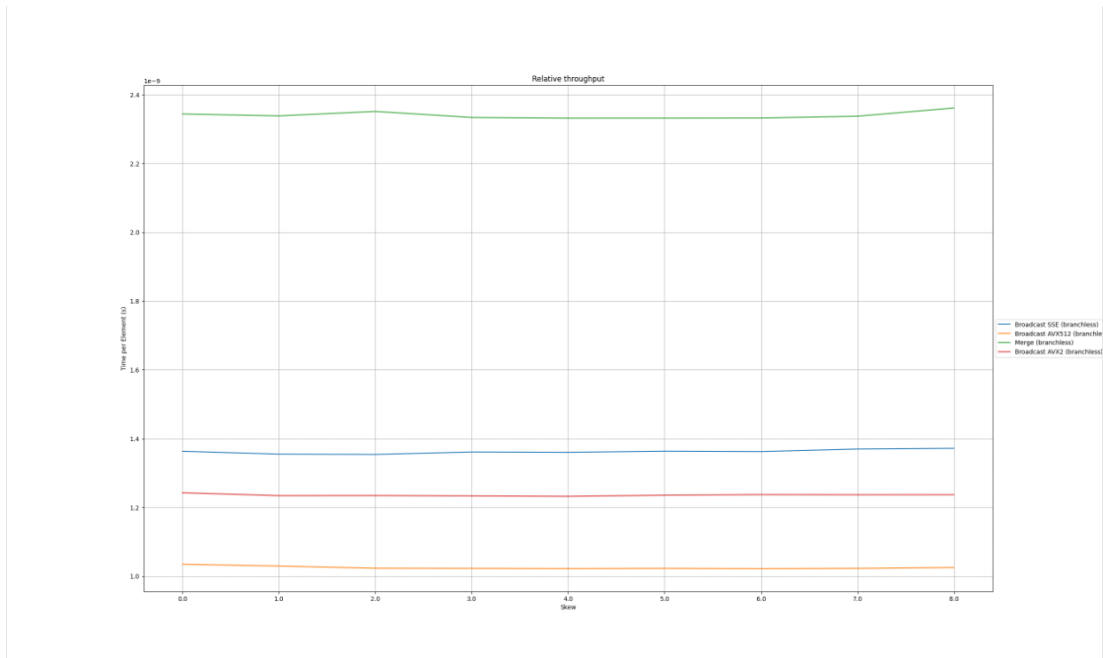


Figure A.4: Arm Broadcast results with varying skew

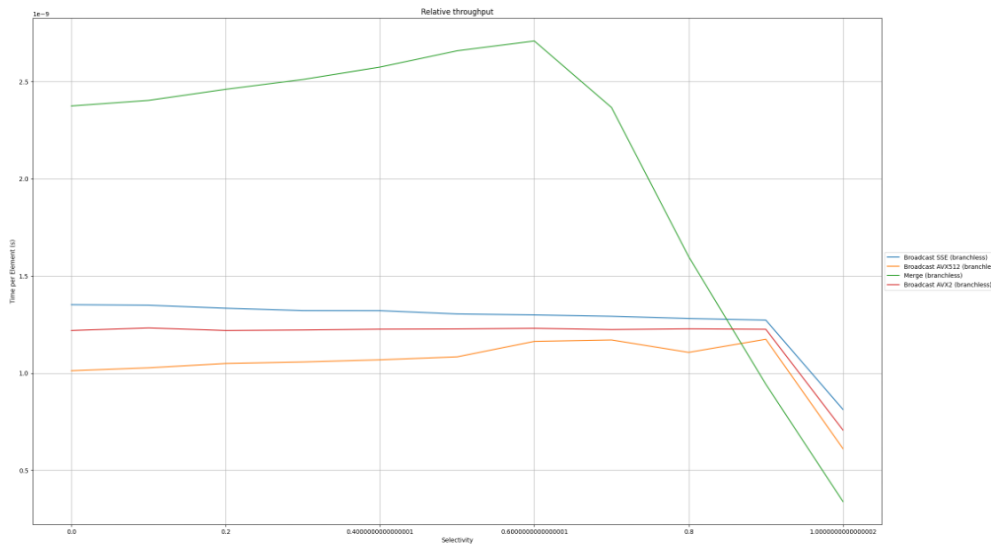


Figure A.5: Arm Broadcast results with varying selectivity

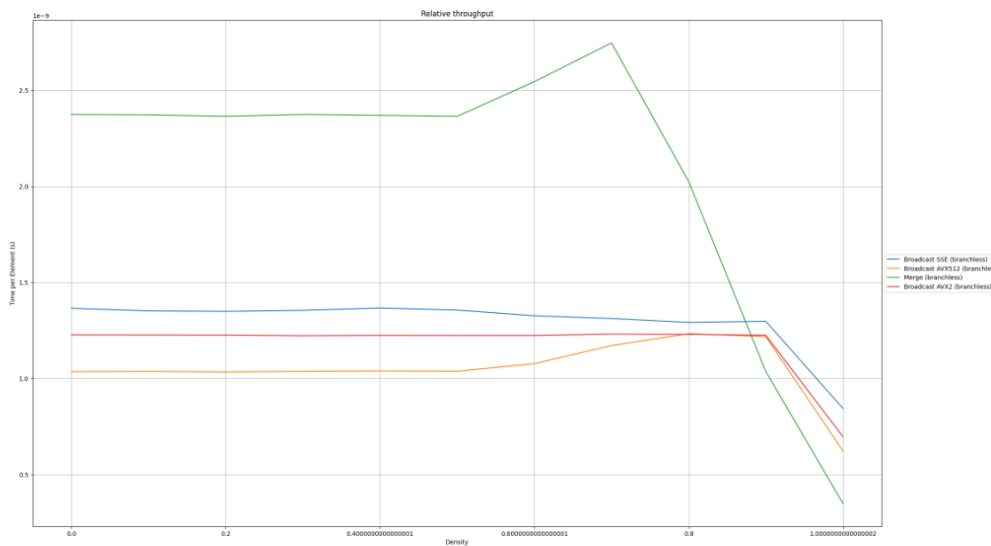


Figure A.6: Arm Broadcast results with varying Density

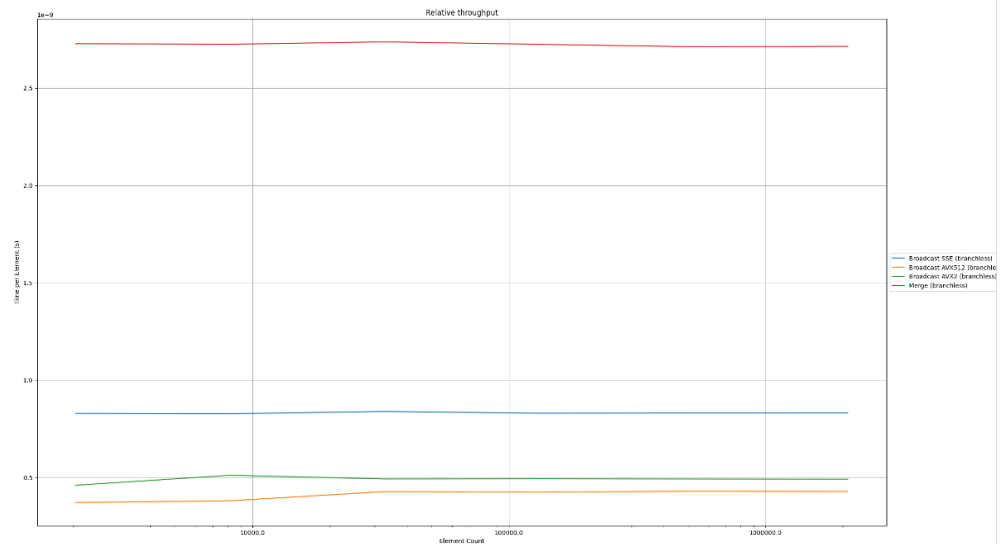


Figure A.7: Intel Xeon Broadcast results with varying set size

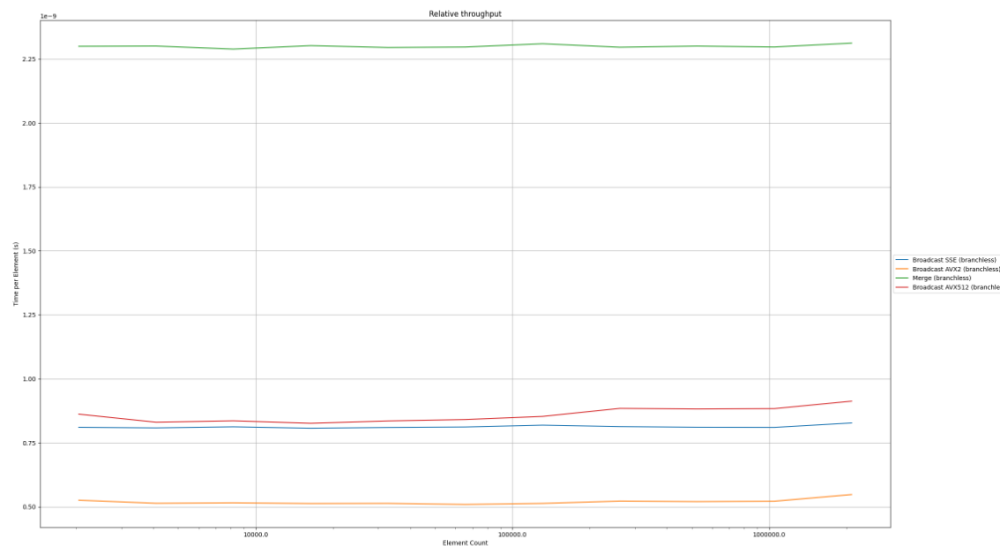


Figure A.8: Intel Core Broadcast results with varying set size

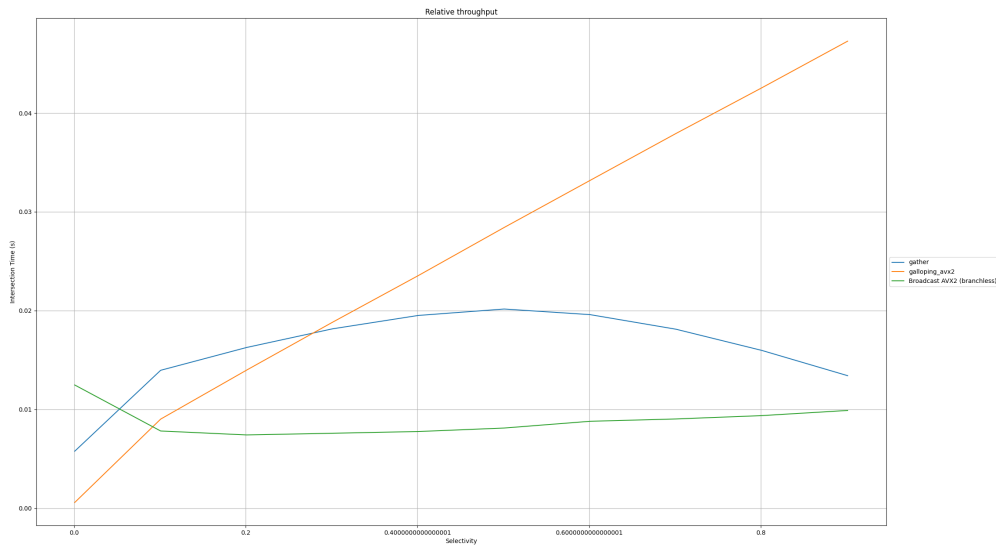


Figure A.9: Gather results against selectivity with a large number of sets

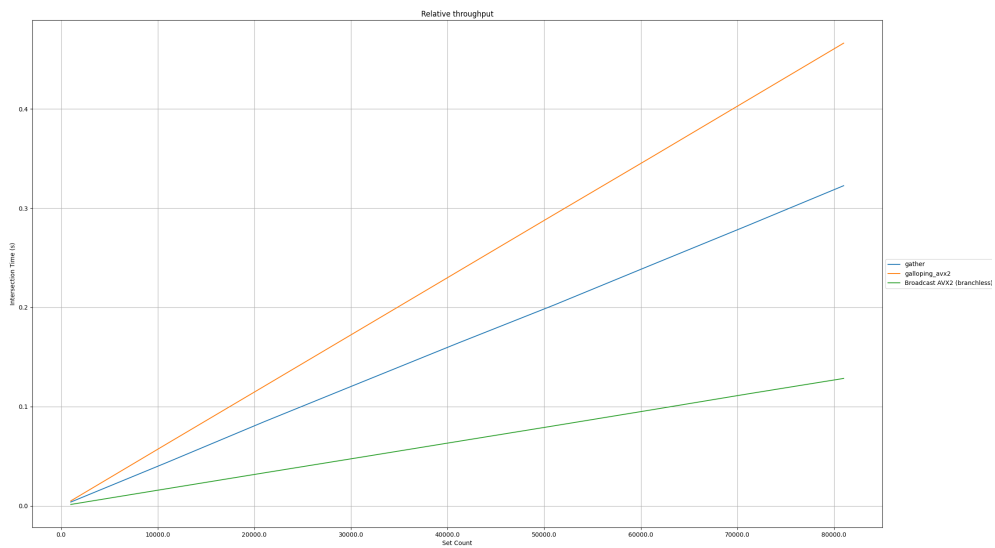


Figure A.10: Gather results against set count with a large number of sets

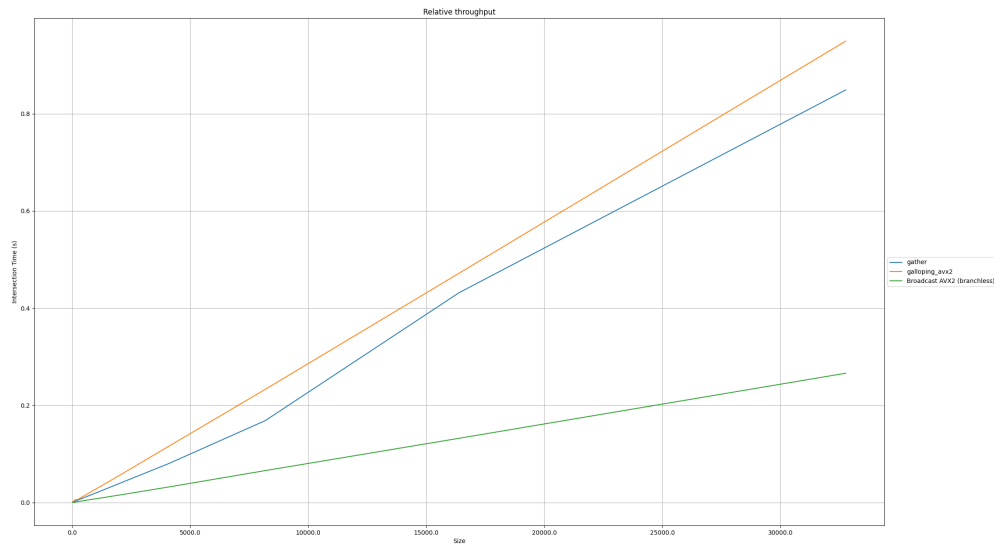


Figure A.11: Gather results against set size with a large number of sets

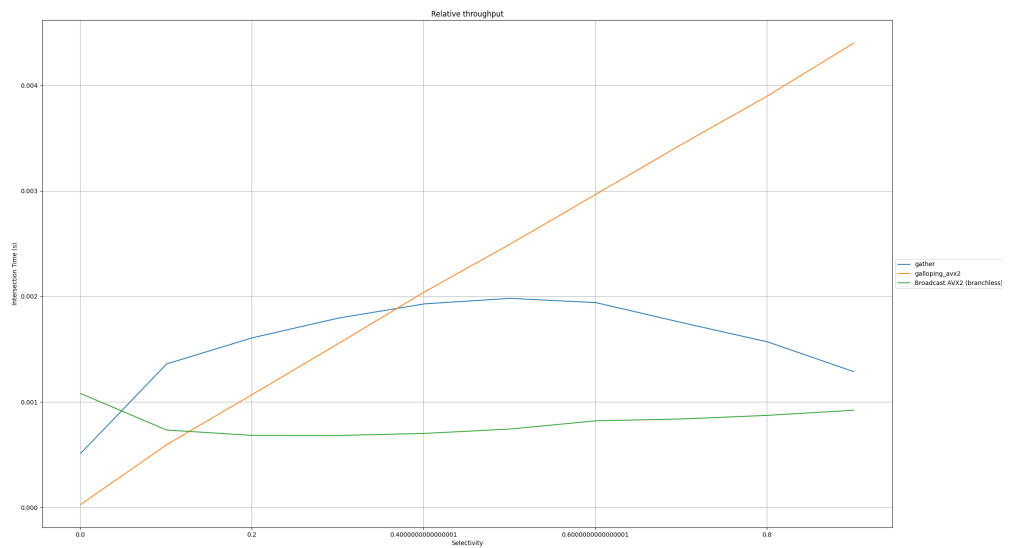


Figure A.12: Gather results against selectivity with a small number of sets

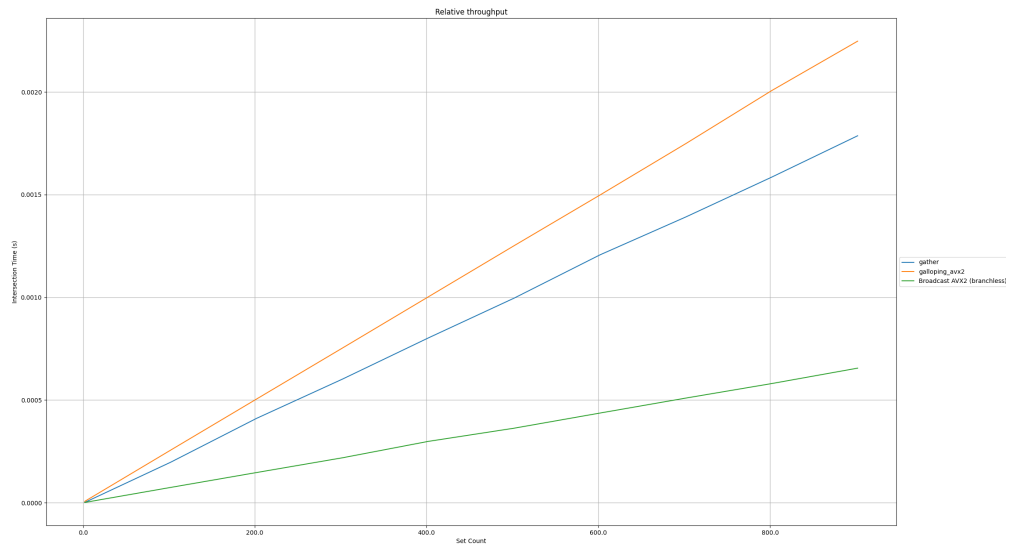


Figure A.13: Gather results against set count with a small number of sets

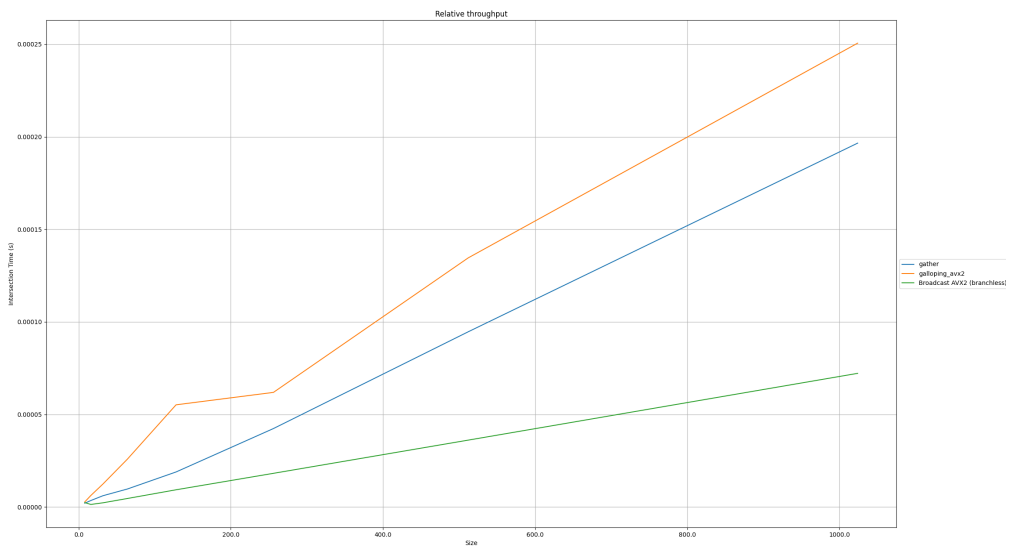


Figure A.14: Gather results against set size with a small number of sets

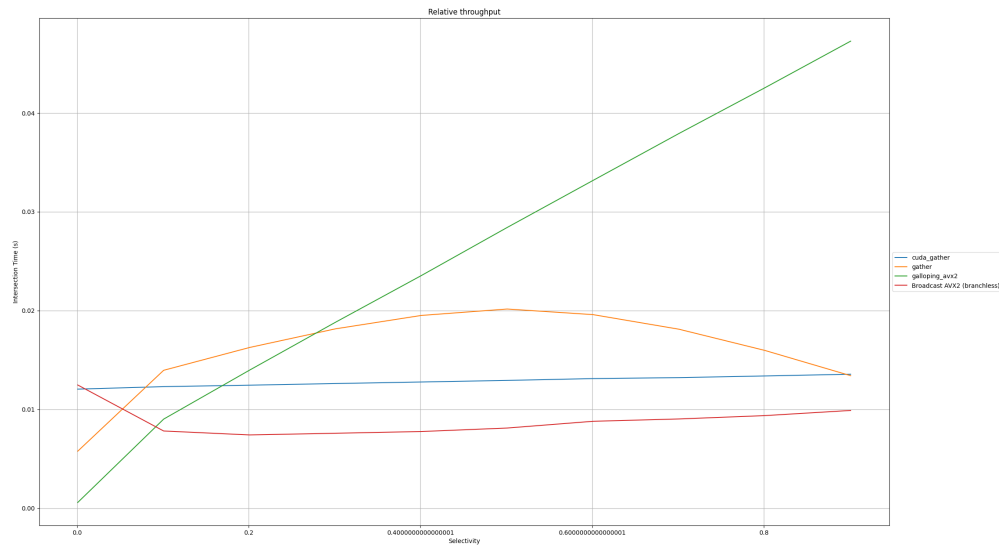


Figure A.15: Cuda Gather results against selectivity with a large number of sets

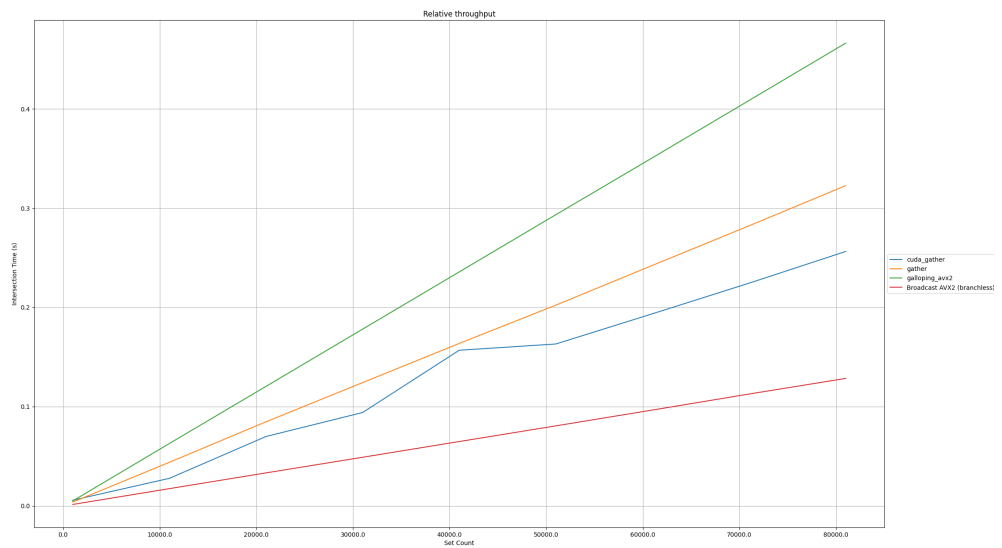


Figure A.16: Cuda Gather results against set count with a large number of sets

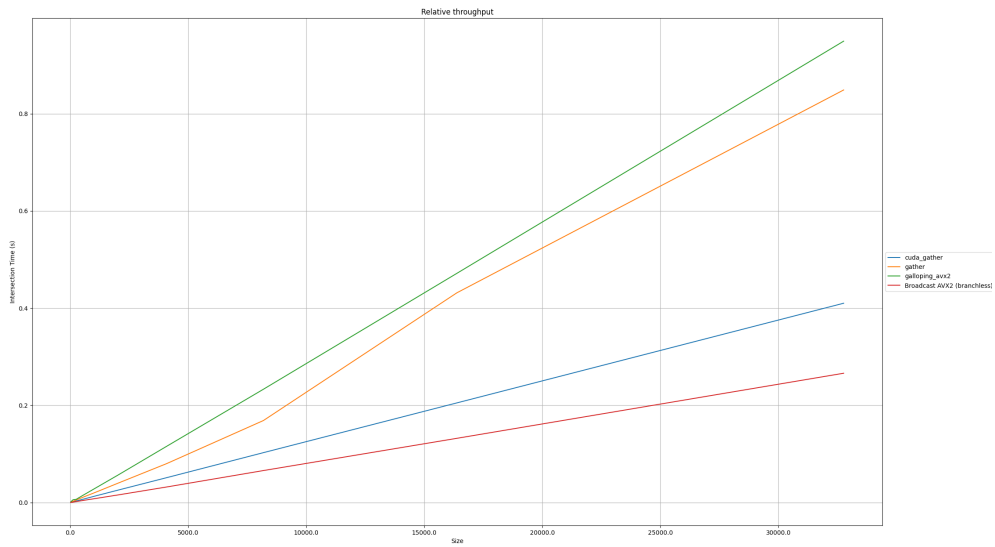


Figure A.17: Cuda Gather results against set size with a large number of sets

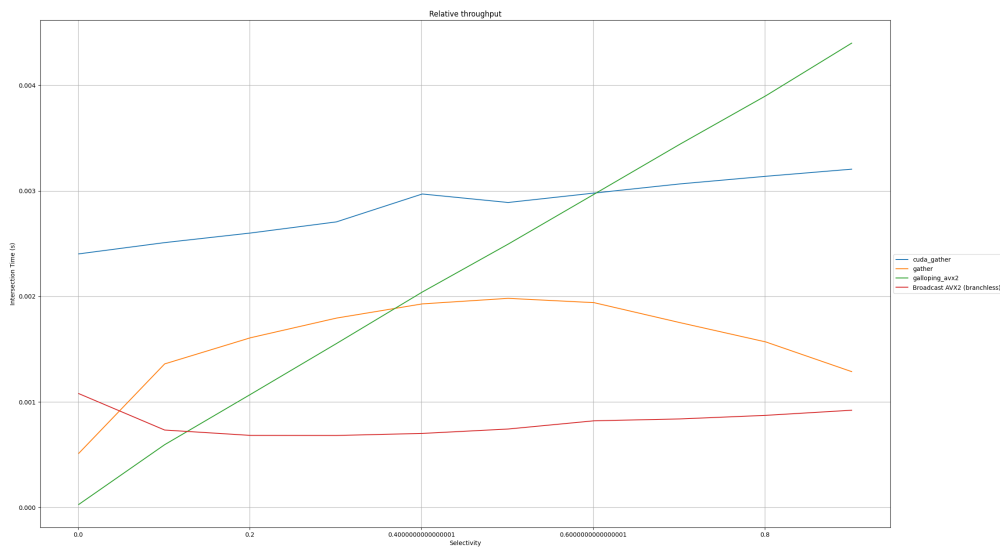


Figure A.18: Cuda Gather results against selectivity with a small number of sets

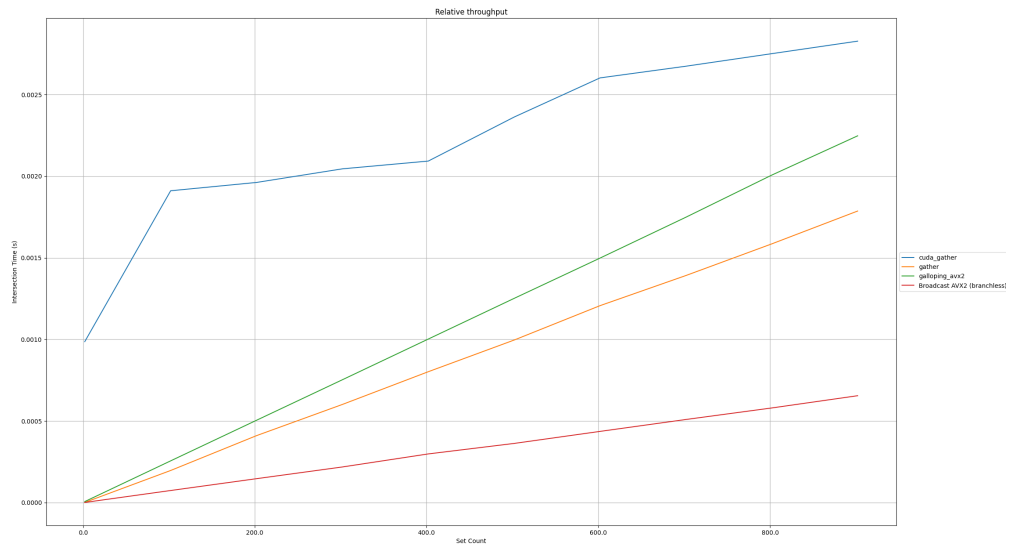


Figure A.19: Cuda Gather results against set count with a small number of sets

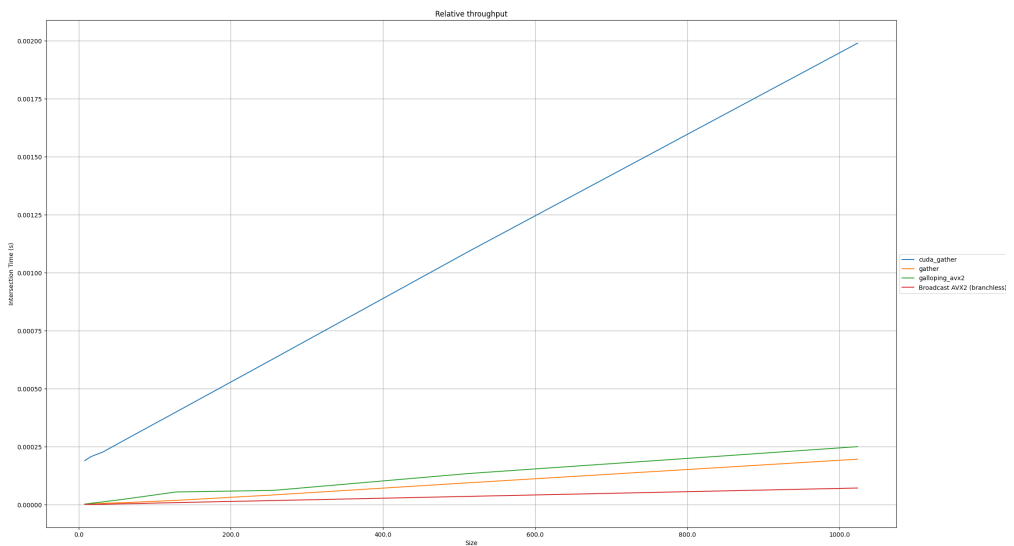


Figure A.20: Cuda Gather results against set size with a small number of sets